CHAPTER

19

TESTING OBJECT-ORIENTED APPLICATIONS

KEY CONCEPTS

In Chapter 18, I noted that the objective of testing, stated simply, is to find the greatest possible number of errors with a manageable amount of effort applied over a realistic time span. Although this fundamental objective remains unchanged for object-oriented software, the nature of OO programs changes both testing strategy and testing tactics.

It could be argued that as reusable class libraries grow in size, greater reuse will mitigate the need for heavy testing of OO systems. Exactly the opposite is true. Binder [Bin94b] discusses this when he states:

> [E]ach reuse is a new context of usage and retesting is prudent. It seems likely that more, not less testing will be needed to obtain high reliability in object-oriented systems.

QUICK LOOK

What is it? The architecture of object-oriented (OO) software results in a series of layered subsystems that encapsulate collaborating classes. Each of these system elements (subsystems and classes) performs functions that help to achieve system requirements. It is necessary to test an OO system at a variety of different levels in an effort to uncover errors that may occur as classes collaborate with one another and subsystems communicate across architectural layers.

Who does it? Object-oriented testing is performed by software engineers and testing specialists.

Why is it important? You have to execute the program before it gets to the customer with the specific intent of removing all errors, so that the customer will not experience the frustration associated with a poor-quality product. In order to find the highest possible number of errors, tests must be conducted systematically and test cases must be designed using disciplined techniques.

What are the steps? OO testing is strategically analogous to the testing of conventional systems, but it is tactically different. Because the OO analysis and design models are similar in structure and content to the resultant OO program, "testing" is initiated with the review of these models. Once code has been generated, OO testing begins "in the small" with class testing. A series of tests are designed that exercise class operations and examine whether errors exist as one class collaborates with other classes. As classes are integrated to form a subsystem, thread-based, use-based, and cluster testing along with fault-based approaches are applied to fully exercise collaborating classes. Finally, use cases (developed as part of the requirements model) are used to uncover errors at the software validation level.

What is the work product? A set of test cases, designed to exercise classes, their collaborations, and behaviors is designed and documented; expected results are defined, and actual results are recorded.

How do I ensure that I've done it right? When you begin testing, change your point of view. Try hard to "break" the software! Design test cases in a disciplined fashion, and review the tests cases you do create for thoroughness.

scenario-based testing 520
thread-based testing 517
use-based testing 517

To adequately test OO systems, three things must be done: (1) the definition of testing must be broadened to include error discovery techniques applied to object-oriented analysis and design models, (2) the strategy for unit and integration testing must change significantly, and (3) the design of test cases must account for the unique characteristics of OO software.

19.1 Broadening the View of Testing

The construction of object-oriented software begins with the creation of requirements (analysis) and design models.[1] Because of the evolutionary nature of the OO software engineering paradigm, these models begin as relatively informal representations of system requirements and evolve into detailed models of classes, class relationships, system design and allocation, and object design (incorporating a model of object connectivity via messaging). At each stage, the models can be "tested" in an attempt to uncover errors prior to their propagation to the next iteration.

It can be argued that the review of OO analysis and design models is especially useful because the same semantic constructs (e.g., classes, attributes, operations, messages) appear at the analysis, design, and code levels. Therefore, a problem in the definition of class attributes that is uncovered during analysis will circumvent side effects that might occur if the problem were not discovered until design or code (or even the next iteration of analysis).

Although the review of the OO analysis and design models is an integral part of "testing" an OO application, recognize that it is not sufficient in and of itself. You must conduct executable tests as well.

For example, consider a class in which a number of attributes are defined during the first iteration of analysis. An extraneous attribute is appended to the class (due to a misunderstanding of the problem domain). Two operations are then specified to manipulate the attribute. A review is conducted and a domain expert points out the problem. By eliminating the extraneous attribute at this stage, the following problems and unnecessary effort may be avoided during analysis:

1. Special subclasses may have been generated to accommodate the unnecessary attribute or exceptions to it. Work involved in the creation of unnecessary subclasses has been avoided.
2. A misinterpretation of the class definition may lead to incorrect or extraneous class relationships.
3. The behavior of the system or its classes may be improperly characterized to accommodate the extraneous attribute.

If the problem is not uncovered during analysis and propagated further, the following problems could occur (and will have been avoided because of the earlier review) during design:

1. Improper allocation of the class to subsystem and/or tasks may occur during system design.

1 Analysis and design modeling techniques are presented in Part 2 of this book. Basic OO concepts are presented in Appendix 2.

2. Unnecessary design work may be expended to create the procedural design for the operations that address the extraneous attribute.
3. The messaging model will be incorrect (because messages must be designed for the operations that are extraneous).

Quote:

"The tools we use have a profound (and devious!) influence on our thinking habits, and, therefore, on our thinking abilities."

Edsger Dijkstra

If the problem remains undetected during design and passes into the coding activity, considerable effort will be expended to generate code that implements an unnecessary attribute, two unnecessary operations, messages that drive interobject communication, and many other related issues. In addition, testing of the class will absorb more time than necessary. Once the problem is finally uncovered, modification of the system must be carried out with the ever-present potential for side effects that are caused by change.

During latter stages of their development, object-oriented analysis (OOA) and design (OOD) models provide substantial information about the structure and behavior of the system. For this reason, these models should be subjected to rigorous review prior to the generation of code.

All object-oriented models should be tested (in this context, the term *testing* incorporates technical reviews) for correctness, completeness, and consistency within the context of the model's syntax, semantics, and pragmatics [Lin94a].

19.2 Testing OOA and OOD Models

Analysis and design models cannot be tested in the conventional sense, because they cannot be executed. However, technical reviews (Chapter 15) can be used to examine their correctness and consistency.

19.2.1 Correctness of OOA and OOD Models

The notation and syntax used to represent analysis and design models will be tied to the specific analysis and design methods that are chosen for the project. Hence syntactic correctness is judged on proper use of the symbology; each model is reviewed to ensure that proper modeling conventions have been maintained.

During analysis and design, you can assess semantic correctness based on the model's conformance to the real-world problem domain. If the model accurately reflects the real world (to a level of detail that is appropriate to the stage of development at which the model is reviewed), then it is semantically correct. To determine whether the model does, in fact, reflect real-world requirements, it should be presented to problem domain experts who will examine the class definitions and hierarchy for omissions and ambiguity. Class relationships (instance connections) are evaluated to determine whether they accurately reflect real-world object connections.[2]

2 Use cases can be invaluable in tracking analysis and design models against real-world usage scenarios for the OO system.

19.2.2 Consistency of Object-Oriented Models

The consistency of object-oriented models may be judged by "considering the relationships among entities in the model. An inconsistent analysis or design model has representations in one part that are not correctly reflected in other portions of the model" [McG94].

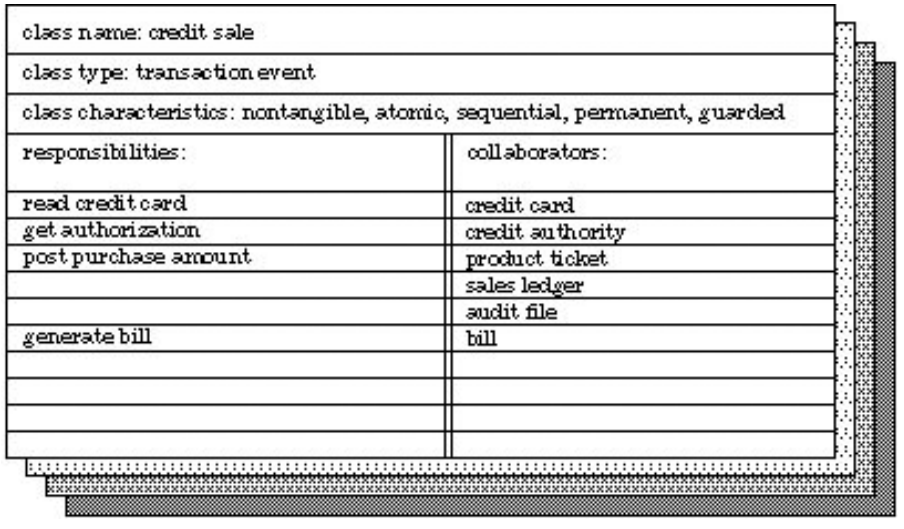
To assess consistency, you should examine each class and its connections to other classes. The class-responsibility-collaboration (CRC) model or an object-relationship diagram can be used to facilitate this activity. As you learned in Chapter 6, the CRC model is composed of CRC index cards. Each CRC card lists the class name, its responsibilities (operations), and its collaborators (other classes to which it sends messages and on which it depends for the accomplishment of its responsibilities). The collaborations imply a series of relationships (i.e., connections) between classes of the OO system. The object relationship model provides a graphic representation of the connections between classes. All of this information can be obtained from the analysis model (Chapters 6 and 7).

To evaluate the class model the following steps have been recommended [McG94]:

1. **Revisit the CRC model and the object-relationship model.** Cross-check to ensure that all collaborations implied by the requirements model are properly reflected in the both.
2. **Inspect the description of each CRC index card to determine if a delegated responsibility is part of the collaborator's definition.** For example, consider a class defined for a point-of-sale checkout system and called **CreditSale.** This class has a CRC index card as illustrated in Figure 19.1.

FIGURE 19.1

An example CRC index card used for review



For this collection of classes and collaborations, ask whether a responsibility (e.g., *read credit card*) is accomplished if delegated to the named collaborator (**CreditCard**). That is, does the class **CreditCard** have an operation that enables it to be read? In this case the answer is "yes." The object-relationship is traversed to ensure that all such connections are valid.

3. **Invert the connection to ensure that each collaborator that is asked for service is receiving requests from a reasonable source.** For example, if the **CreditCard** class receives a request for **purchase amount** from the **CreditSale** class, there would be a problem. **CreditCard** does not know the purchase amount.
4. **Using the inverted connections examined in step 3, determine whether other classes might be required or whether responsibilities are properly grouped among the classes.**
5. **Determine whether widely requested responsibilities might be combined into a single responsibility.** For example, *read credit card* and *get authorization* occur in every situation. They might be combined into a *validate credit request* responsibility that incorporates getting the credit card number and gaining authorization.

You should apply steps 1 through 5 iteratively to each class and through each evolution of the requirements model.

Once the design model (Chapters 9 through 11) is created, you should also conduct reviews of the system design and the object design. The system design depicts the overall product architecture, the subsystems that compose the product, the manner in which subsystems are allocated to processors, the allocation of classes to subsystems, and the design of the user interface. The object model presents the details of each class and the messaging activities that are necessary to implement collaborations between classes.

The system design is reviewed by examining the object-behavior model developed during object-oriented analysis and mapping required system behavior against the subsystems designed to accomplish this behavior. Concurrency and task allocation are also reviewed within the context of system behavior. The behavioral states of the system are evaluated to determine which exist concurrently. Use cases are used to exercise the user interface design.

The object model should be tested against the object-relationship network to ensure that all design objects contain the necessary attributes and operations to implement the collaborations defined for each CRC index card. In addition, the detailed specification of operation details (i.e., the algorithms that implement the operations) is reviewed.

19.3 Object-Oriented Testing Strategies

As I noted in Chapter 18, the classical software testing strategy begins with "testing in the small" and works outward toward "testing in the large." Stated in the jargon of software testing (Chapter 18), you begin with *unit testing,* then progress toward *integration testing,* and culminate with *validation and system testing.* In conventional applications, unit testing focuses on the smallest compilable program unit—the subprogram (e.g., component, module, subroutine, procedure). Once each of these units has been testing individually, it is integrated into a program structure while a series of regression tests are run to uncover errors due to interfacing the modules and side effects that are caused by the addition of new units. Finally, the system as a whole is tested to ensure that errors in requirements are uncovered.

19.3.1 Unit Testing in the OO Context

The smallest testable "unit" in OO software is the class. Class testing is driven by the operations encapsulated by the class and the state behavior of the class.

When object-oriented software is considered, the concept of the unit changes. Encapsulation drives the definition of classes and objects. This means that each class and each instance of a class (object) packages attributes (data) and the operations (also known as methods or services) that manipulate these data. Rather than testing an individual module, the smallest testable unit is the encapsulated class. Because a class can contain a number of different operations and a particular operation may exist as part of a number of different classes, the meaning of unit testing changes dramatically.

We can no longer test a single operation in isolation (the conventional view of unit testing) but rather, as part of a class. To illustrate, consider a class hierarchy in which an operation *X()* is defined for the superclass and is inherited by a number of subclasses. Each subclass uses operation *X()*, but it is applied within the context of the private attributes and operations that have been defined for each subclass. Because the context in which operation *X()* is used varies in subtle ways, it is necessary to test operation *X()* in the context of each of the subclasses. This means that testing operation *X()* in a vacuum (the traditional unit-testing approach) is ineffective in the object-oriented context.

Class testing for OO software is the equivalent of unit testing for conventional software.[3] Unlike unit testing of conventional software, which tends to focus on the algorithmic detail of a module and the data that flows across the module interface, class testing for OO software is driven by the operations encapsulated by the class and the state behavior of the class.

19.3.2 Integration Testing in the OO Context

Because object-oriented software does not have a hierarchical control structure, conventional top-down and bottom-up integration strategies have little meaning.

3 Test-case design methods for OO classes are discussed in Sections 19.4 through 19.6.

In addition, integrating operations one at a time into a class (the conventional incremental integration approach) is often impossible because of the "direct and indirect interactions of the components that make up the class" [Ber93].

KEY POINT
Integration testing for OO software tests a set of classes that are required to respond to a given event.

There are two different strategies for integration testing of OO systems [Bin94a]. The first, *thread-based testing,* integrates the set of classes required to respond to one input or event for the system. Each thread is integrated and tested individually. Regression testing is applied to ensure that no side effects occur. The second integration approach, *use-based testing,* begins the construction of the system by testing those classes (called *independent classes*) that use very few (if any) of server classes. After the independent classes are tested, the next layer of classes, called *dependent classes,* that use the independent classes are tested. This sequence of testing layers of dependent classes continues until the entire system is constructed. Unlike conventional integration, the use of driver and stubs (Chapter 18) as replacement operations is to be avoided, when possible.

Cluster testing [McG94] is one step in the integration testing of OO software. Here, a cluster of collaborating classes (determined by examining the CRC and object-relationship model) is exercised by designing test cases that attempt to uncover errors in the collaborations.

19.3.3 Validation Testing in an OO Context

At the validation or system level, the details of class connections disappear. Like conventional validation, the validation of OO software focuses on user-visible actions and user-recognizable outputs from the system. To assist in the derivation of validation tests, the tester should draw upon use cases (Chapters 5 and 6) that are part of the requirements model. The use case provides a scenario that has a high likelihood of uncovered errors in user-interaction requirements.

Conventional black-box testing methods (Chapter 18) can be used to drive validation tests. In addition, you may choose to derive test cases from the object-behavior model and from an event flow diagram created as part of OOA.

19.4 Object-Oriented Testing Methods

Quote:
"I see testers as the bodyguards of the project. We defend our developer's flank from failure, while they focus on creating success."
James Bach

The architecture of object-oriented software results in a series of layered subsystems that encapsulate collaborating classes. Each of these system elements (subsystems and classes) performs functions that help to achieve system requirements. It is necessary to test an OO system at a variety of different levels in an effort to uncover errors that may occur as classes collaborate with one another and subsystems communicate across architectural layers.

Test-case design methods for object-oriented software continue to evolve. However, an overall approach to OO test-case design has been suggested by Berard [Ber93]:

1. Each test case should be uniquely identified and explicitly associated with the class to be tested.

2. The purpose of the test should be stated.
3. A list of testing steps should be developed for each test and should contain:
   a. A list of specified states for the class that is to be tested
   b. A list of messages and operations that will be exercised as a consequence of the test
   c. A list of exceptions that may occur as the class is tested
   d. A list of external conditions (i.e., changes in the environment external to the software that must exist in order to properly conduct the test)
   e. Supplementary information that will aid in understanding or implementing the test

Unlike conventional test-case design, which is driven by an input-process-output view of software or the algorithmic detail of individual modules, object-oriented testing focuses on designing appropriate sequences of operations to exercise the states of a class.

19.4.1 The Test-Case Design Implications of OO Concepts

WebRef
An excellent collection of papers and resources on OO testing can be found at **www.rbsc.com**.

As a class evolves through the requirements and design models, it becomes a target for test-case design. Because attributes and operations are encapsulated, testing operations outside of the class is generally unproductive. Although encapsulation is an essential design concept for OO, it can create a minor obstacle when testing. As Binder [Bin94a] notes, "Testing requires reporting on the concrete and abstract state of an object." Yet, encapsulation can make this information somewhat difficult to obtain. Unless built-in operations are provided to report the values for class attributes, a snapshot of the state of an object may be difficult to acquire.

Inheritance may also present you with additional challenges during test-case design. I have already noted that each new usage context requires retesting, even though reuse has been achieved. In addition, multiple inheritance[4] complicates testing further by increasing the number of contexts for which testing is required [Bin94a]. If subclasses instantiated from a superclass are used within the same problem domain, it is likely that the set of test cases derived for the superclass can be used when testing the subclass. However, if the subclass is used in an entirely different context, the superclass test cases will have little applicability and a new set of tests must be designed.

19.4.2 Applicability of Conventional Test-Case Design Methods

The white-box testing methods described in Chapter 18 can be applied to the operations defined for a class. Basis path, loop testing, or data flow techniques can help to ensure that every statement in an operation has been tested. However, the concise

4 An OO concept that should be used with extreme care.

structure of many class operations causes some to argue that the effort applied to white-box testing might be better redirected to tests at a class level.

Black-box testing methods are as appropriate for OO systems as they are for systems developed using conventional software engineering methods. As I noted in Chapter 18, use cases can provide useful input in the design of black-box and state-based tests.

KEY POINT

The strategy for fault-based testing is to hypothesize a set of plausible faults and then derive tests to prove each hypothesis.

19.4.3 Fault-Based Testing[5]

The object of *fault-based testing* within an OO system is to design tests that have a high likelihood of uncovering plausible faults. Because the product or system must conform to customer requirements, preliminary planning required to perform fault-based testing begins with the analysis model. The tester looks for plausible faults (i.e., aspects of the implementation of the system that may result in defects). To determine whether these faults exist, test cases are designed to exercise the design or code.

Of course, the effectiveness of these techniques depends on how testers perceive a plausible fault. If real faults in an OO system are perceived to be implausible, then this approach is really no better than any random testing technique. However, if the analysis and design models can provide insight into what is likely to go wrong, then fault-based testing can find significant numbers of errors with relatively low expenditures of effort.

? What types of faults are encountered in operation calls and message connections?

Integration testing looks for plausible faults in operation calls or message connections. Three types of faults are encountered in this context: unexpected result, wrong operation/message used, and incorrect invocation. To determine plausible faults as functions (operations) are invoked, the behavior of the operation must be examined.

Integration testing applies to attributes as well as to operations. The "behaviors" of an object are defined by the values that its attributes are assigned. Testing should exercise the attributes to determine whether proper values occur for distinct types of object behavior.

It is important to note that integration testing attempts to find errors in the client object, not the server. Stated in conventional terms, the focus of integration testing is to determine whether errors exist in the calling code, not the called code. The operation call is used as a clue, a way to find test requirements that exercise the calling code.

19.4.4 Test Cases and the Class Hierarchy

Inheritance does not obviate the need for thorough testing of all derived classes. In fact, it can actually complicate the testing process. Consider the following situation. A class **Base** contains operations *inherited()* and *redefined()*. A class **Derived** redefines *redefined()* to serve in a local context. There is little doubt that **Derived::redefined()**

5 Sections 19.4.3 through 19.4.6 have been adapted from an article by Brian Marick posted on the Internet newsgroup comp.testing. This adaptation is included with the permission of the author. For further information on these topics, see [Mar94]. It should be noted that the techniques discussed in Sections 19.4.3 through 19.4.6 are also applicable for conventional software.

has to be tested because it represents a new design and new code. But does **Derived::inherited()** have to be retested?

If **Derived::inherited()** calls *redefined()* and the behavior of *redefined()* has changed, **Derived::inherited()** may mishandle the new behavior. Therefore, it needs new tests even though the design and code have not changed. It is important to note, however, that only a subset of all tests for **Derived::inherited()** may have to be conducted. If part of the design and code for *inherited()* does not depend on *redefined()* (i.e., that does not call it nor call any code that indirectly calls it), that code need not be retested in the derived class.

KEY POINT
Even though a base class has been thoroughly tested, you will still have to test all classes derived from it.

Base::redefined() and **Derived::redefined()** are two different operations with different specifications and implementations. Each would have a set of test requirements derived from the specification and implementation. Those test requirements probe for plausible faults: integration faults, condition faults, boundary faults, and so forth. But the operations are likely to be similar. Their sets of test requirements will overlap. The better the OO design, the greater is the overlap. New tests need to be derived only for those **Derived::redefined()** requirements that are not satisfied by the **Base::redefined()** tests.

To summarize, the **Base::redefined()** tests are applied to objects of class **Derived.** Test inputs may be appropriate for both base and derived classes, but the expected results may differ in the derived class.

19.4.5 Scenario-Based Test Design

Fault-based testing misses two main types of errors: (1) incorrect specifications and (2) interactions among subsystems. When errors associated with an incorrect specification occur, the product doesn't do what the customer wants. It might do the wrong thing or omit important functionality. But in either circumstance, quality (conformance to requirements) suffers. Errors associated with subsystem interaction occur when the behavior of one subsystem creates circumstances (e.g., events, data flow) that cause another subsystem to fail.

KEY POINT
Scenario-based testing will uncover errors that occur when any actor interacts with the software.

Scenario-based testing concentrates on what the user does, not what the product does. This means capturing the tasks (via use cases) that the user has to perform and then applying them and their variants as tests.

Scenarios uncover interaction errors. But to accomplish this, test cases must be more complex and more realistic than fault-based tests. Scenario-based testing tends to exercise multiple subsystems in a single test (users do not limit themselves to the use of one subsystem at a time).

As an example, consider the design of scenario-based tests for a text editor by reviewing the use cases that follow:

Use Case: Fix the Final Draft

Background: It's not unusual to print the "final" draft, read it, and discover some annoying errors that weren't obvious from the on-screen image. This use case describes the sequence of events that occurs when this happens.

1. Print the entire document.
2. Move around in the document, changing certain pages.
3. As each page is changed, it's printed.
4. Sometimes a series of pages is printed.

Quote:

"If you want and expect a program to work, you will more likely see a working program—you will miss failures."

Cem Kaner et al.

This scenario describes two things: a test and specific user needs. The user needs are obvious: (1) a method for printing single pages and (2) a method for printing a range of pages. As far as testing goes, there is a need to test editing after printing (as well as the reverse). Therefore, you work to design tests that will uncover errors in the editing function that were caused by the printing function; that is, errors that will indicate that the two software functions are not properly independent.

Use Case: Print a New Copy

Background: Someone asks the user for a fresh copy of the document. It must be printed.

1. Open the document.
2. Print it.
3. Close the document.

Again, the testing approach is relatively obvious. Except that this document didn't appear out of nowhere. It was created in an earlier task. Does that task affect this one?

In many modern editors, documents remember how they were last printed. By default, they print the same way next time. After the **Fix the Final Draft** scenario, just selecting "Print" in the menu and clicking the Print button in the dialog box will cause the last corrected page to print again. So, according to the editor, the correct scenario should look like this:

Although scenario-based testing has merit, you will get a higher return on time invested by reviewing use cases when they are developed as part of the analysis model.

Use Case: Print a New Copy

1. Open the document.
2. Select "Print" in the menu.
3. Check if you're printing a page range; if so, click to print the entire document.
4. Click on the Print button.
5. Close the document.

But this scenario indicates a potential specification error. The editor does not do what the user reasonably expects it to do. Customers will often overlook the check noted in step 3. They will then be annoyed when they trot off to the printer and find one page when they wanted 100. Annoyed customers signal specification bugs.

You might miss this dependency as you design tests, but it is likely that the problem would surface during testing. You would then have to contend with the probable response, "That's the way it's supposed to work!"

19.4.6 Testing Surface Structure and Deep Structure

Surface structure refers to the externally observable structure of an OO program. That is, the structure that is immediately obvious to an end user. Rather than performing functions, the users of many OO systems may be given objects to manipulate in some way. But whatever the interface, tests are still based on user tasks. Capturing these tasks involves understanding, watching, and talking with representative users (and as many nonrepresentative users as are worth considering).

Testing surface structure is analogous to black-box testing. Deep structure testing is similar to white-box testing.

There will surely be some difference in detail. For example, in a conventional system with a command-oriented interface, the user might use the list of all commands as a testing checklist. If no test scenarios existed to exercise a command, testing has likely overlooked some user tasks (or the interface has useless commands). In an object-based interface, the tester might use the list of all objects as a testing checklist.

The best tests are derived when the designer looks at the system in a new or unconventional way. For example, if the system or product has a command-based interface, more thorough tests will be derived if the test-case designer pretends that operations are independent of objects. Ask questions like, "Might the user want to use this operation—which applies only to the **Scanner** object—while working with the printer?" Whatever the interface style, test-case design that exercises the surface structure should use both objects and operations as clues leading to overlooked tasks.

"Be not ashamed of mistakes and thus make them crimes."

Confucius

Deep structure refers to the internal technical details of an OO program, that is, the structure that is understood by examining the design and/or code. Deep structure testing is designed to exercise dependencies, behaviors, and communication mechanisms that have been established as part of the design model for OO software.

The requirements and design models are used as the basis for deep structure testing. For example, the UML collaboration diagram or the deployment model depicts collaborations between objects and subsystems that may not be externally visible. The test-case design then asks: "Have we captured (as a test) some task that exercises the collaboration noted on the collaboration diagram? If not, why not?"

19.5 Testing Methods Applicable at the Class Level

The number of possible permutations for random testing can grow quite large. A strategy similar to orthogonal array testing can be used to improve testing efficiency.

Testing "in the small" focuses on a single class and the methods that are encapsulated by the class. Random testing and partitioning are methods that can be used to exercise a class during OO testing.

19.5.1 Random Testing for OO Classes

To provide brief illustrations of these methods, consider a banking application in which an **Account** class has the following operations: *open(), setup(), deposit(), withdraw(), balance(), summarize(), creditLimit(),* and *close()* [Kir94]. Each of these operations may be applied for **Account,** but certain constraints (e.g., the account must be opened before other operations can be applied and closed after all operations are

completed) are implied by the nature of the problem. Even with these constraints, there are many permutations of the operations. The minimum behavioral life history of an instance of **Account** includes the following operations:

open • setup • deposit • withdraw • close

This represents the minimum test sequence for account. However, a wide variety of other behaviors may occur within this sequence:

open • setup • deposit • [deposit | withdraw | balance | summarize | creditLimit]^n • withdraw • close

A variety of different operation sequences can be generated randomly. For example:

Test case r_1: **open • setup • deposit • deposit • balance • summarize • withdraw • close**

Test case r_2: **open • setup • deposit • withdraw • deposit • balance • creditLimit • withdraw • close**

These and other random order tests are conducted to exercise different class instance life histories.

SafeHome

Class Testing

The scene: Shakira's cubicle.

The players: Jamie and Shakira—members of the *SafeHome* software engineering team who are working on test-case design for the security function.

The conversation:

Shakira: I've developed some tests for the **Detector** class [Figure 10.4]—you know, the one that allows access to all of the **Sensor** objects for the security function. You familiar with it?

Jamie (laughing): Sure, it's the one that allowed you to add the "doggie angst" sensor.

Shakira: The one and only. Anyway, it has an interface with four ops: *read(), enable(), disable(),* and *test()*. Before a sensor can be read, it must be enabled. Once it's enabled, it can be read and tested. It can be disabled at any time, except if an alarm condition is being processed. So I defined a simple test sequence that will exercise its behavioral life history. [Shows Jamie the following sequence.]

#1: enable•test•read•disable

Jamie: That'll work, but you've got to do more testing than that!

Shakira: I know, I know, here are some other sequences I've come up with. [Shows Jamie the following sequences.]

#2: enable•test*[read]^n•test•disable

#3: [read]^n

#4: enable*disable•[test I read]

Jamie: So let me see if I understand the intent of these. #1 goes through a normal life history, sort of a conventional usage. #2 repeats the read operation *n* times, and that's a likely scenario. #3 tries to read the sensor before it's been enabled . . . that should produce an error message of some kind, right? #4 enables and disables the sensor and then tries to read it. Isn't that the same as test #2?

Shakira: Actually no. In #4, the sensor has been enabled. What #4 really tests is whether the disable op works as it should. A *read()* or *test()* after *disable()* should generate the error message. If it doesn't, then we have an error in the disable op.

Jamie: Cool. Just remember that the four tests have to be applied for every sensor type since all the ops may be subtly different depending on the type of sensor.

Shakira: Not to worry. That's the plan.

19.5.2 Partition Testing at the Class Level

? What testing options are available at the class level?

Partition testing reduces the number of test cases required to exercise the class in much the same manner as equivalence partitioning (Chapter 18) for traditional software. Input and output are categorized and test cases are designed to exercise each category. But how are the partitioning categories derived?

State-based partitioning categorizes class operations based on their ability to change the state of the class. Again considering the **Account** class, state operations include *deposit()* and *withdraw(),* whereas nonstate operations include *balance(), summarize(),* and *creditLimit().* Tests are designed in a way that exercises operations that change state and those that do not change state separately. Therefore,

Test case p_1: **open • setup • deposit • deposit • withdraw • withdraw • close**
Test case p_2: **open • setup • deposit • summarize • creditLimit • withdraw • close**

Test case p_1 changes state, while test case p_2 exercises operations that do not change state (other than those in the minimum test sequence).

Attribute-based partitioning categorizes class operations based on the attributes that they use. For the **Account** class, the attributes **balance** and **creditLimit** can be used to define partitions. Operations are divided into three partitions: (1) operations that use **creditLimit,** (2) operations that modify **creditLimit,** and (3) operations that do not use or modify **creditLimit.** Test sequences are then designed for each partition.

Category-based partitioning categorizes class operations based on the generic function that each performs. For example, operations in the **Account** class can be categorized in initialization operations (*open, setup*), computational operations (*deposit, withdraw*), queries (*balance, summarize, creditLimit*), and termination operations (*close*).

19.6 Interclass Test-Case Design

Quote:

"The boundary that defines the scope of unit and integration testing is different for object-oriented development. Tests can be designed and exercised at many points in the process. Thus 'design a little, code a little' becomes 'design a little, code a little, test a little.'"

Robert Binder

Test-case design becomes more complicated as integration of the object-oriented system begins. It is at this stage that testing of collaborations between classes must begin. To illustrate "interclass test-case generation" [Kir94], we expand the banking example introduced in Section 19.5 to include the classes and collaborations noted in Figure 19.2. The direction of the arrows in the figure indicates the direction of messages, and the labeling indicates the operations that are invoked as a consequence of the collaborations implied by the messages.

Like the testing of individual classes, class collaboration testing can be accomplished by applying random and partitioning methods, as well as scenario-based testing and behavioral testing.

19.6.1 Multiple Class Testing

Kirani and Tsai [Kir94] suggest the following sequence of steps to generate multiple class random test cases:

1. For each client class, use the list of class operations to generate a series of random test sequences. The operations will send messages to other server classes.

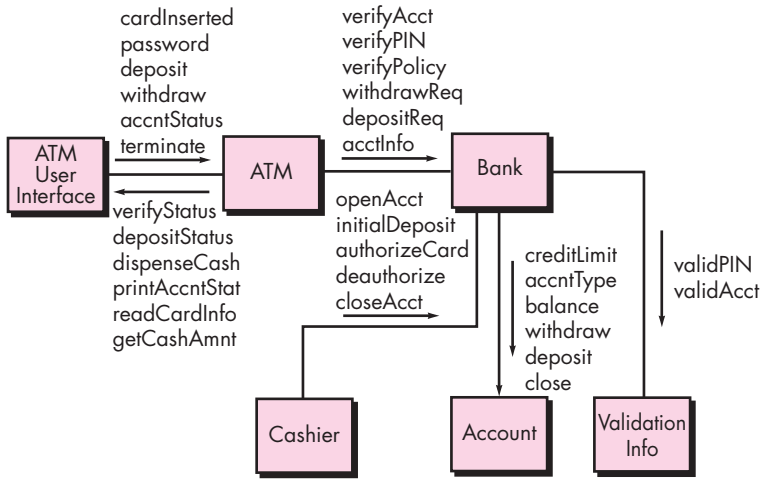


FIGURE 19.2
Class collaboration diagram for banking application
Source: Adapted from [Kir94].

2. For each message that is generated, determine the collaborator class and the corresponding operation in the server object.
3. For each operation in the server object (that has been invoked by messages sent from the client object), determine the messages that it transmits.
4. For each of the messages, determine the next level of operations that are invoked and incorporate these into the test sequence.

To illustrate [Kir94], consider a sequence of operations for the **Bank** class relative to an **ATM** class (Figure 19.2):

verifyAcct • verifyPIN • [[verifyPolicy • withdrawReq] | depositReq | acctInfoREQ]^n

A random test case for the **Bank** class might be

Test case r_3 = **verifyAcct • verifyPIN • depositReq**

In order to consider the collaborators involved in this test, the messages associated with each of the operations noted in test case r_3 are considered. **Bank** must collaborate with **ValidationInfo** to execute the *verifyAcct()* and *verifyPIN().* **Bank** must collaborate with **Account** to execute *depositReq().* Hence, a new test case that exercises these collaborations is

Test case r_4 = **verifyAcct [Bank:validAcctValidationInfo] • verifyPIN [Bank: validPinValidationInfo] • depositReq [Bank: depositaccount]**

The approach for multiple class partition testing is similar to the approach used for partition testing of individual classes. A single class is partitioned as discussed in Section 19.5.2. However, the test sequence is expanded to include those operations that are invoked via messages to collaborating classes. An alternative approach partitions tests based on the interfaces to a particular class. Referring to Figure 19.2,

the **Bank** class receives messages from the **ATM** and **Cashier** classes. The methods within **Bank** can therefore be tested by partitioning them into those that serve **ATM** and those that serve **Cashier.** State-based partitioning (Section 19.5.2) can be used to refine the partitions further.

19.6.2 Tests Derived from Behavior Models

The use of the state diagram as a model that represents the dynamic behavior of a class is discussed in Chapter 7. The state diagram for a class can be used to help derive a sequence of tests that will exercise the dynamic behavior of the class (and those classes that collaborate with it). Figure 19.3 [Kir94] illustrates a state diagram for the **Account** class discussed earlier. Referring to the figure, initial transitions move through the *empty acct* and *setup acct* states. The majority of all behavior for instances of the class occurs while in the *working acct* state. A final withdrawal and account closure cause the account class to make transitions to the *nonworking acct* and *dead acct* states, respectively.

The tests to be designed should achieve coverage of every state. That is, the operation sequences should cause the **Account** class to make transition through all allowable states:

Test case s_1: **open • setupAccnt • deposit (initial) • withdraw (final) • close**

It should be noted that this sequence is identical to the minimum test sequence discussed in Section 19.5.2. Adding additional test sequences to the minimum sequence,

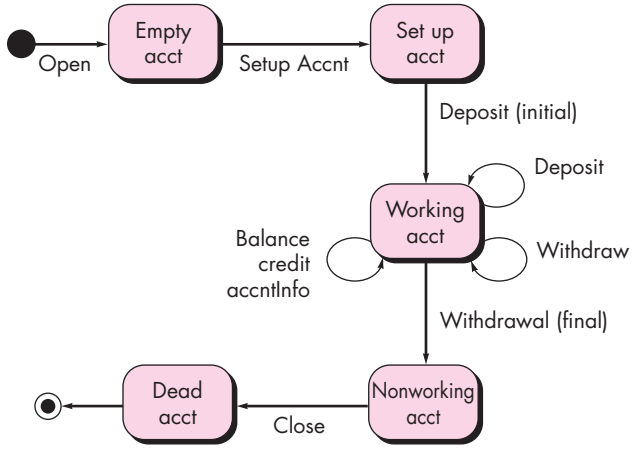
Test case s_2: **open • setupAccnt • deposit(initial) • deposit • balance • credit • withdraw (final) • close**

Test case s_3: **open • setupAccnt • deposit(initial) • deposit • withdraw • accntInfo • withdraw (final) • close**

FIGURE 19.3

State diagram for the Account class

Source: Adapted from [Kir94].



Still more test cases could be derived to ensure that all behaviors for the class have been adequately exercised. In situations in which the class behavior results in a collaboration with one or more classes, multiple state diagrams are used to track the behavioral flow of the system.

The state model can be traversed in a "breadth-first" [McG94] manner. In this context, breadth-first implies that a test case exercises a single transition and that when a new transition is to be tested only previously tested transitions are used.

Consider a **CreditCard** object that is part of the banking system. The initial state of **CreditCard** is *undefined* (i.e., no credit card number has been provided). Upon reading the credit card during a sale, the object takes on a *defined* state; that is, the attributes **card number** and **expiration date,** along with bank-specific identifiers are defined. The credit card is *submitted* when it is sent for authorization, and it is *approved* when authorization is received. The transition of **CreditCard** from one state to another can be tested by deriving test cases that cause the transition to occur. A breadth-first approach to this type of testing would not exercise *submitted* before it exercised *undefined* and *defined.* If it did, it would make use of transitions that had not been previously tested and would therefore violate the breadth-first criterion.

19.7 SUMMARY

The overall objective of object-oriented testing—to find the maximum number of errors with a minimum amount of effort is identical to the objective of conventional software testing. But the strategy and tactics for OO testing differ significantly. The view of testing broadens to include the review of both the requirements and design model. In addition, the focus of testing moves away from the procedural component (the module) and toward the class.

Because the OO requirements and design models and the resulting source code are semantically coupled, testing (in the form of technical reviews) begins during the modeling activity. For this reason, the review of CRC, object-relationship, and object-behavior models can be viewed as first-stage testing.

Once code is available, unit testing is applied for each class. The design of tests for a class uses a variety of methods: fault-based testing, random testing, and partition testing. Each of these methods exercise the operations encapsulated by the class. Test sequences are designed to ensure that relevant operations are exercised. The state of the class, represented by the values of its attributes, is examined to determine if errors exist.

Integration testing can be accomplished using a thread-based or use-based strategy. Thread-based testing integrates the set of classes that collaborate to respond to one input or event. Use-based testing constructs the system in layers, beginning with those classes that do not make use of server classes. Integration test-case design methods can also make use of random and partition tests. In addition, scenario-based testing and tests derived from behavioral models can be used to test a class

and its collaborators. A test sequence tracks the flow of operations across class collaborations.

OO system validation testing is black-box oriented and can be accomplished by applying the same black-box methods discussed for conventional software. However, scenario-based testing dominates the validation of OO systems, making the use case a primary driver for validation testing.

Problems and Points to Ponder

19.1. In your own words, describe why the class is the smallest reasonable unit for testing within an OO system.

19.2. Why do we have to retest subclasses that are instantiated from an existing class, if the existing class has already been thoroughly tested? Can we use the test-case design for the existing class?

19.3. Why should "testing" begin with object-oriented analysis and design?

19.4. Derive a set of CRC index cards for *SafeHome,* and conduct the steps noted in Section 19.2.2 to determine if inconsistencies exist.

19.5. What is the difference between thread-based and use-based strategies for integration testing? How does cluster testing fit in?

19.6. Apply random testing and partitioning to three classes defined in the design for the *SafeHome* system. Produce test cases that indicate the operation sequences that will be invoked.

19.7. Apply multiple class testing and tests derived from the behavioral model for the *SafeHome* design.

19.8. Derive four additional tests using random testing and partitioning methods as well as multiple class testing and tests derived from the behavioral model for the banking application presented in Sections 19.5 and 19.6.

Further Readings and Information Sources

Many books on testing noted in the *Further Readings* sections of Chapters 17 and 18 discuss testing of OO systems to some extent. Schach (*Object-Oriented and Classical Software Engineering,* McGraw-Hill, 6th ed., 2004) considers OO testing within the context of broader software engineering practice. Sykes and McGregor (*Practical Guide to Testing Object-Oriented Software,* Addison-Wesley, 2001), Bashir and Goel (*Testing Object-Oriented Software,* Springer 2000), Binder (*Testing Object-Oriented Systems,* Addison-Wesley, 1999), and Kung and his colleagues (*Testing Object-Oriented Software,* Wiley-IEEE Computer Society Press, 1998) treat OO testing in significant detail.

A wide variety of information sources on object-oriented testing methods is available on the Internet. An up-to-date list of World Wide Web references that are relevant to testing techniques can be found at the SEPA website: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**.